

Estrutura de Dados 2

Prof. Igor Calebe Zadi
igor.zadi@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo
Campus Catanduva



I. Complexidade de Algoritmos



I. Fundamentos de Estruturas de Dados

1. Introdução à análise de algoritmos
2. Crescimento de funções
3. Notações assintóticas
4. Análise de algoritmos iterativos e recursivos
5. Pior, melhor e caso médio
6. Classes de complexidade: P, NP
7. Tabelas de crescimento e desempenho

1. Introdução à análise de algoritmos



1. Introdução à análise de algoritmos

- Motivação
 - Por que analisar a complexidade de algoritmos?



1. Introdução à análise de algoritmos

- Exemplo:
 - Partindo da sequencia de Fibonacci
 - 1, 1, 2, 3, 5, 8, 13, 21...

1. Introdução à análise de algoritmos

Algoritmo 1

Algoritmo Fib (n)

Entrada: n , inteiro, $n \geq 1$.

Saída: retorna o elemento de ordem n da seqüência de Fibonacci.

```
{  
    a := 1; aa := 1; f := 1; i := 3;  
    enquanto ( i ≤ n )  
    {  
        aux := f; f := a + aa; aa := a; a := aux;  
        i := i + 1  
    }  
  
    retornar f  
}
```

1. Introdução à análise de algoritmos

• Algoritmo 2

Algoritmo Fib (n)

Entrada: n , inteiro, $n \geq 1$.

Saída: retorna o elemento de ordem n da seqüência de Fibonacci.

```
{  
    se ( $n \leq 2$ )  
        retornar 1  
    senão  
        retornar Fib ( $n - 1$ ) + Fib ( $n - 2$ )  
}
```




1. Introdução à análise de algoritmos

- Qual deles tem a melhor performance na execução?
 - “Qual irá executar mais rápido?”
- Utilizando uma calculadora, em quanto tempo você calcularia o número de ordem 100 da série de Fibonacci?

1. Introdução à análise de algoritmos

- Em um computador com as configurações:
 - **24 núcleos totais** (8 Performance + 16 Efficiency)
 - **Frequência base:** 3,2 GHz
 - **Frequência turbo:** até 6,0 GHz (para os núcleos de desempenho)
 - Para estimar o **número teórico máximo de operações por segundo**, consideramos:
 - Pico de 6 GHz = 6×10^9 ciclos/s por núcleo
 - Supondo 1 operação por ciclo (ideal)
 - Com 24 núcleos. temos:

$$\text{Ops/s} = 24 \times 6 \times 10^9 = 144 \times 10^9 = \boxed{1,44 \times 10^{11}}$$



1. Introdução à análise de algoritmos

- Este computador executará:
 - O algoritmo 1 em **alguns milissegundos**
 - O algoritmo 2: aproximadamente **279,2 bilhões de anos!!**

1. Introdução à análise de algoritmos

- Divagando:

- O supercomputador Frontier (2025) que executa **1,1 quintilhão** de operações por segundo ($1,1 \times 10^{18}$)

n	2^n	Tempo (s)	Tempo Aproximado
10	1.024	$9,3 \times 10^{-16}$ s	quase instantâneo
20	1.048.576	$9,5 \times 10^{-13}$ s	quase instantâneo
30	1.073.741.824	$9,76 \times 10^{-10}$ s	menos de 1 microssegundo
40	$1,10 \times 10^{12}$	$1,00 \times 10^{-6}$ s	~1 microssegundo
50	$1,13 \times 10^{15}$	$1,03 \times 10^{-3}$ s	~1 milissegundo
100	$1,27 \times 10^{30}$	$1,15 \times 10^{12}$ s	~36.500 anos



1. Introdução à análise de algoritmos

Mesmo o computador mais poderoso do mundo não consegue vencer um algoritmo ruim.

A escolha do algoritmo certo faz mais diferença do que ter o computador mais rápido.



1. Introdução à análise de algoritmos

- Motivação
 - Por que analisar a complexidade de algoritmos?
 - é fundamental para projetar algoritmos eficientes.
 - podemos desenvolver um algoritmo e depois analisar a sua complexidade para verificar a sua eficiência.



1. Introdução à análise de algoritmos

- A análise de algoritmos tem como função **determinar os recursos necessários** para executar um dado algoritmo
- Ela estuda a **correção e o desempenho** através da análise de correção e da análise de complexidade
- Dados dois algoritmos para um mesmo problema, a análise permite **decidir qual é o mais eficiente.**



1. Introdução à análise de algoritmos

- Analisar um algoritmo
 - Investigar e definir o comportamento:
 - tempo;
 - de espaço (consumo de memória);
 - outras propriedades.



1. Introdução à análise de algoritmos

- Análise de **correção**
 - A análise da correção procura entender porque um dado algoritmo funciona
 - É preciso ter certeza de que o algoritmo está correto
 - Como algoritmos prometem resolver problemas, é esperado que ele cumpra essa premissa.



1. Introdução à análise de algoritmos

- Análise de **complexidade**
 - Procura estimar a **velocidade** do algoritmo
 - **Prever o tempo** que o algoritmo vai consumir
 - Verificar **qual é o custo** de usar um dado algoritmo para resolver um problema específico.

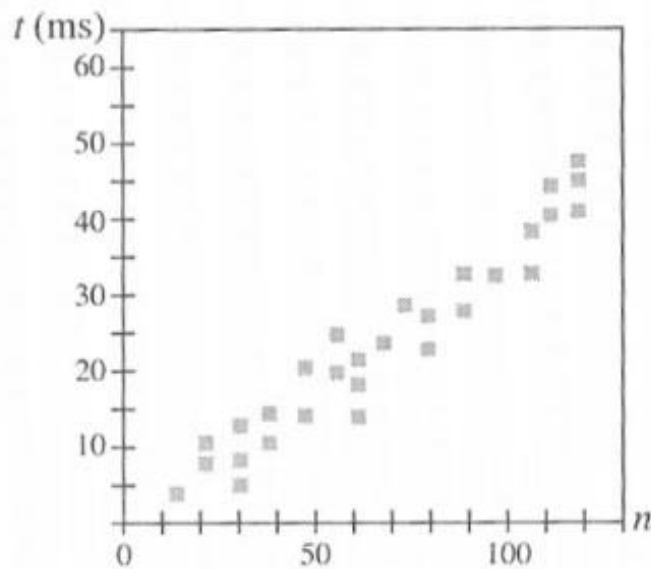


1. Introdução à análise de algoritmos

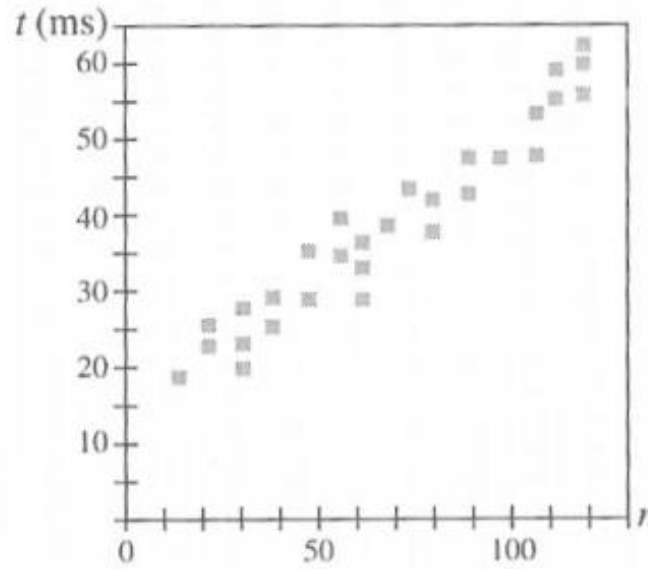
- Metodologias para analisar algoritmos
 - Experimental (empírica)
 - Teórica

1. Introdução à análise de algoritmos

- Experimental
 - Mede-se o tempo (t) de execução do algoritmo por um computador para diferentes tamanhos de entrada (n).



(a)



(b)



1. Introdução à análise de algoritmos

- Experimental
 - executando o programa com *inputs* de tamanho e composição variados, mas:
 - nem sempre é simples concretizar o algoritmo, ou
 - os resultados podem não ser conclusivos,
 - comparação de algoritmos obriga a usar *software e hardware iguais*.



1. Introdução à análise de algoritmos

- Teoria da complexidade de algoritmos
 - Modelo de computação (abstrato, não se prende a um computador ou tecnologia).
 - Objetivo: calcular uma função matemática $f(n)$ que descreve o comportamento do algoritmo considerando um modelo de computação (n representa o tamanho da entrada).

1. Introdução à análise de algoritmos

- Baseando-se nos exemplos anteriores:

Algoritmo Fib (n)

Entrada: n , inteiro, $n \geq 1$.

Saída: retorna o elemento de ordem n da sequência de Fibonacci.

```
{  
  a := 1; aa := 1; f := 1; i := 3;  
  enquanto ( i ≤ n )  
  {  
    aux := f; f := a + aa; aa := a; a := aux;  
    i := i + 1  
  }  
  
  retornar f  
}
```

Tempo de execução do Algoritmo 1:
 $T(n) = O(n)$

1. Introdução à análise de algoritmos

- Baseando-se nos exemplos anteriores:

Algoritmo Fib (n)

Entrada: n , inteiro, $n \geq 1$.

Saída: retorna o elemento de ordem n da seqüência de Fibonacci.

```
{  
  se ( $n \leq 2$ )  
    retornar 1  
  senão  
    retornar Fib ( $n - 1$ ) + Fib ( $n - 2$ )  
}
```

Tempo de execução do Algoritmo 2:

$$T(n) = O(2^n)$$



1. Introdução à análise de algoritmos

- Iniciando a análise de um algoritmo:
 - Quantas vezes um laço é executado?

1. Introdução à análise de algoritmos

- Quantas vezes o comando C será executado no trecho de código?

```
i := 1;  
enquanto ( i ≤ 10 )  
{  
    C;  
    i := i + 1;  
}
```

1. Introdução à análise de algoritmos

- E agora?

```
i := 4;  
enquanto ( i ≤ n )  
{  
    C;  
    i := i + 2;  
}
```

1. Introdução à análise de algoritmos

- Fórmulas úteis PA

- Último termo:

$$a_k = a_1 + (k - 1) r.$$

Diagram illustrating the components of the formula for the k-th term of an Arithmetic Progression (PA):

- a_k : Último (Last term)
- a_1 : Primeiro (First term)
- k : Quantidade (Quantity)
- r : Razão (Reason/Reasoning)

- Soma dos k termos da PA:

$$S(k) = a_1 + a_2 + a_3 + \dots + a_k = k (a_k + a_1)/2.$$

1. Introdução à análise de algoritmos

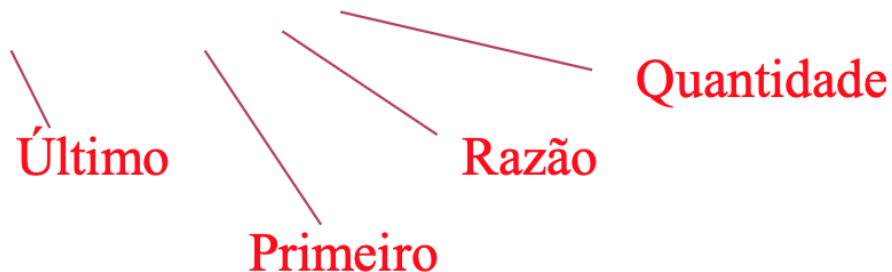
- E agora?

```
i := 1;  
enquanto ( i ≤ n )  
{  
    C;  
    i := i * 2;  
}
```

1. Introdução à análise de algoritmos

- Fórmulas úteis PG

- Último termo: $a_k = a_1 r^{k-1}.$



- Soma dos k termos da PG:

$$S(k) = a_1 + a_2 + a_3 + \dots + a_k = a_1(r^k - 1)/(r - 1).$$

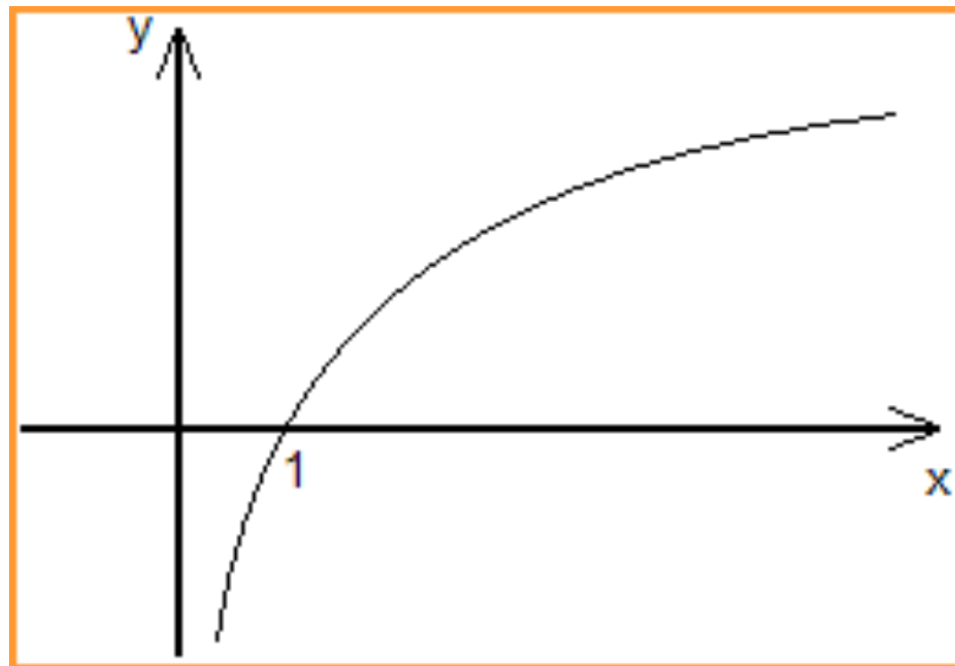
Se $0 < r < 1$ então a soma de infinitos termos da PG é

$$S = a_1 / (1 - r).$$

1. Introdução à análise de algoritmos

- Logaritmos:

$$\log_b x = y \text{ se e somente se } b^y = x.$$



1. Introdução à análise de algoritmos

- Propriedades :

$$\log_c a.b = \log_c a + \log_c b.$$

$$\log_c a / b = \log_c a - \log_c b.$$

$$\log_b a = \frac{1}{\log_a b}.$$

$$\log_a x = \frac{\log_b x}{\log_b a}.$$

$$b^{\log_b x} = x.$$

$$b^{\log_a x} = x^{\log_a b}.$$



1. Introdução à análise de algoritmos

- Modelo de Computação RAM (utilizado na disciplina):
 - Um autômato no qual:
 - instruções são executadas uma após a outra.
Nenhuma
 - operação ocorre em paralelo.
 - um único processador executa as operações.
 - o tempo de acesso é uniforme para todas as localizações de memória.

1. Introdução à análise de algoritmos

- Qual a complexidade de tempo:

Algoritmo Potencia_de_2

Entrada: lê n , inteiro, $n \geq 1$.

Saída: escreve 2^n .

```
{  
    ler ( $n$ );  
     $p := 1$ ;  
    enquanto (  $n > 0$  )  
    {  
         $p := 2 * p$ ;  
         $n := n - 1$   
    }  
    escrever  $p$   
}
```



1. Introdução à análise de algoritmos

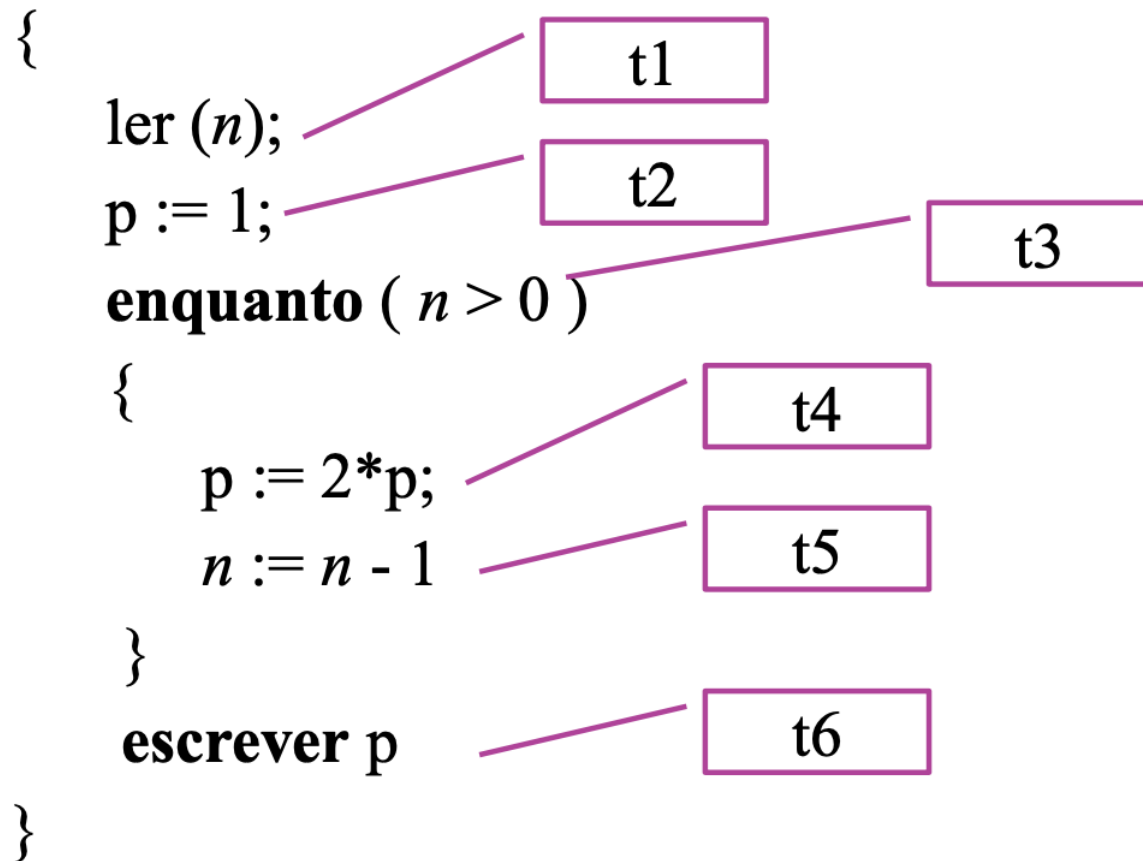
- Qual a complexidade de tempo:
 - Sejam constantes **t1, t2, t3, t4, t5 e t6** representando tempos.

1. Introdução à análise de algoritmos

Algoritmo Potencia_de_2

Entrada: lê n , inteiro, $n \geq 1$.

Saída: escreve 2^n .



Complexidade de tempo $T(n)$.

$$T(n) = t1 + t2 + n(t4 + t5) + (n + 1)t3 + t6$$



1. Introdução à análise de algoritmos

- $T(n) = t_1 + t_2 + t_6 + n t_4 + n t_5 + n t_3 + t_3.$
- $T(n) = t_1 + t_2 + t_6 + t_3 + n (t_4 + t_5 + t_3).$
- Sejam constantes:
 - $c_1 = t_1 + t_2 + t_6 + t_3$ e
 - $c_2 = t_4 + t_5 + t_3.$
- $T(n) = c_1 + c_2 n.$

1. Introdução à análise de algoritmos

- Vendo o código de maneira diferente (RAM):

ler (n);	READ 1
	LOAD \$1
$p := 1$;	STORE 2
enquanto ($n > 0$)	L: LOAD 1
	JPZ F
	LOAD 2
	MUL \$2
$p := 2 * p$;	STORE 2
	LOAD 1;
	SUB \$1
$n := n - 1$	STORE 1
	JP L
escrever (p)	F: WRITE 2
	HALT

1. Introdução à análise de algoritmos

- Entendendo o código:

READ 1	; lê da entrada padrão e armazena na posição de memória 1.
LOAD \$1	; carrega no registrador A a constante 1.
STORE 2	; armazena na posição de memória 2 o valor do registrador A.
L: LOAD 1	; carrega no registrador A o valor da posição de memória 1.
JPZ F	; desvia para a instrução de rótulo “F” se o valor do registrador ; A for igual a zero.
LOAD 2	; carrega no registrador A o valor da posição de memória 2.
MUL \$2	; multiplica o valor do registrador A pela constante 2.
STORE 2	
LOAD 1;	
SUB \$1	; subtrai o valor do registrador pela constante 1.
STORE 1	
JP L	; desvia para a instrução de rótulo “L”.
F: WRITE 2	; escreve na saída padrão o valor da posição de memória 2
HALT	; encerra a execução.

1. Introdução à análise de algoritmos

- Calculando a complexidade:

```
READ 1  
LOAD $1  
STORE 2
```

```
L:  LOAD 1  
    JPZ F
```

Teste
 $n > 0$

```
    LOAD 2  
    MUL $2  
    STORE 2  
    LOAD 1;  
    SUB $1  
    STORE 1  
    JP L
```

```
F:  WRITE 2  
    HALT
```

Suponha que cada instrução executa em um tempo constante igual a t .

Complexidade de tempo $T(n)$.

$$T(n) = 3t +$$

$$7tn +$$

$$2t(n+1) +$$

$$2t$$

$$T(n) = 3t + 7tn + 2tn + 2t + 2t$$

$$T(n) = 7t + 9tn$$

$$T(n) = c_1 + c_2 n,$$

para constantes $c_1 = 7t$ e $c_2 = 9t$.



1. Introdução à análise de algoritmos

- Resultado importante:
 - O tempo de execução de cada comando de em uma linguagem de alto nível é **proporcional** ao tempo de execução de comandos da máquina do modelo RAM.
 - Logo podemos trabalhar sobre algoritmos escritos em linguagem de alto nível **como se estivéssemos trabalhando sobre a máquina do modelo RAM.**



1. Introdução à análise de algoritmos

- Ignorando fatores constantes
 - Nós iremos **ignorar fatores constantes** e nos **concentraremos no comportamento assintótico da função que descreve** a complexidade do algoritmo.
 - Por exemplo, se o algoritmo gasta $100n$ passos para retornar o resultado, então nós diremos que o tempo de execução do algoritmo será descrito por n (i.e., é proporcional a n).

1. Introdução à análise de algoritmos

- Qual a complexidade de espaço?

Algoritmo Busca (A, n, x)

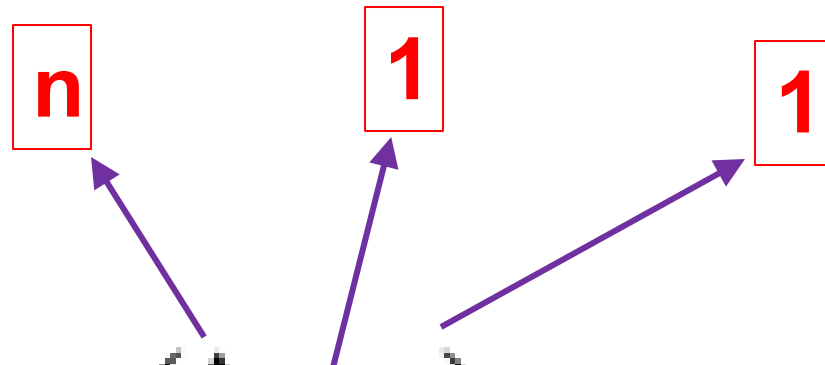
Entrada: procura o valor igual a x no vetor “A”, de “ n ” elementos.

Saída: retorna o índice do elemento do vetor “A” igual a “ x ” ou retorna zero, caso tal elemento não exista.

```
{  
     $i := 1$ ;  
    enquanto (  $i \leq n$  e  $x \neq A[i]$  )  
         $i := i + 1$ ;  
  
    se (  $i \leq n$  )  
        retornar  $i$   
    senão  
        retornar 0  
}
```



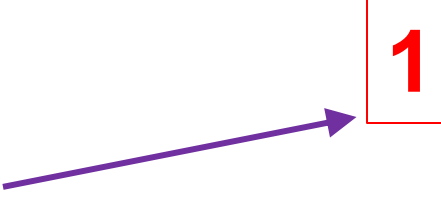
1. Introdução à análise de algoritmos



Algoritmo Busca (A, n, x)

Entrada: procura o valor igual a x no vetor “ A ”, de “ n ” elementos.

Saída: retorna o índice do elemento do vetor “ A ” igual a x ,
caso tal elemento não exista.

{
 $i := 1;$ 



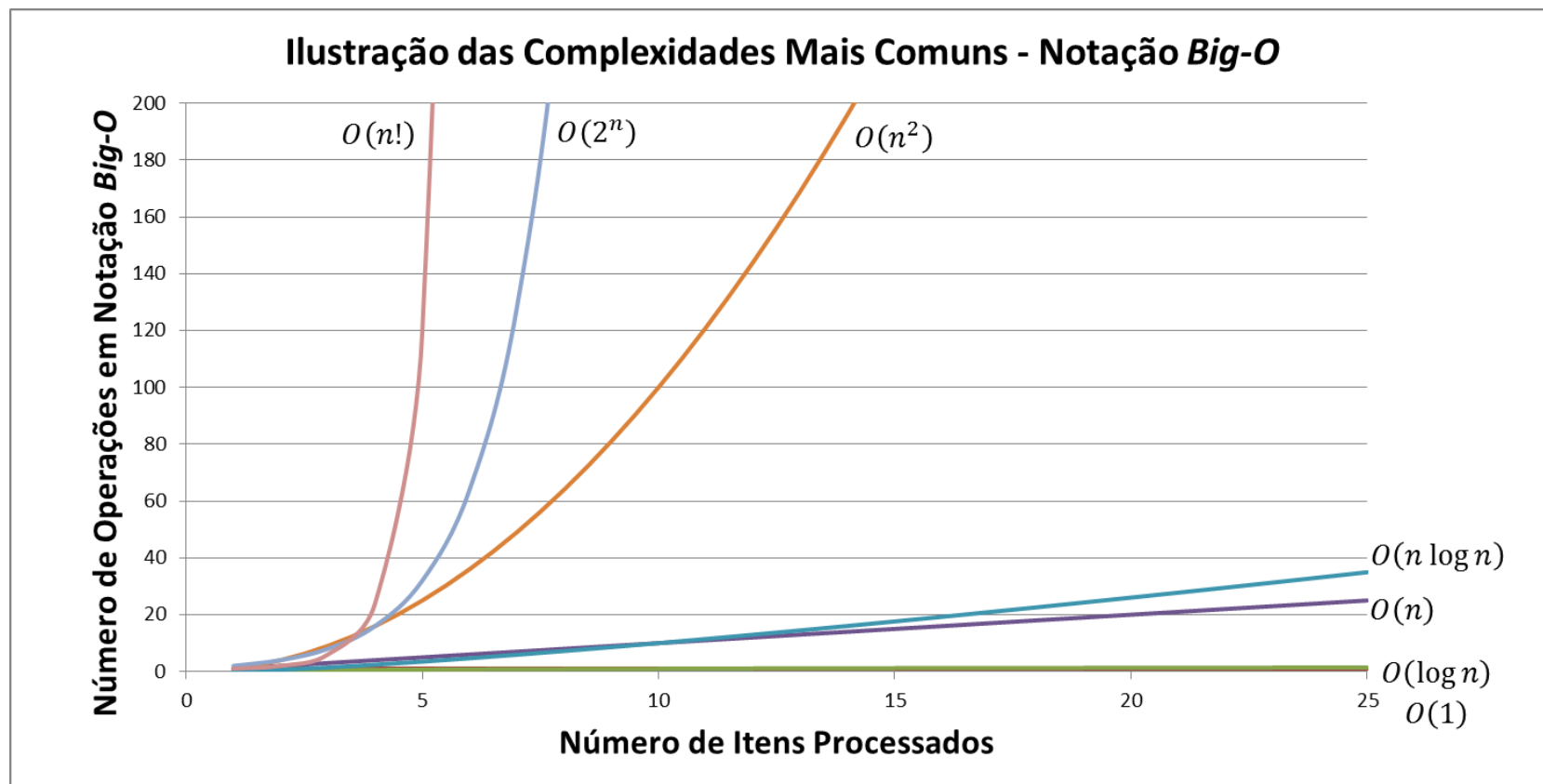
2. Crescimento de Funções

- A ideia central é entender como o tempo ou o espaço usados por um algoritmo aumentam com o tamanho da entrada (n)
- Pequenas diferenças na complexidade têm grande impacto para entradas grandes
- Exemplos: $O(n)$ vs. $O(n^2)$ para $n = 1000$

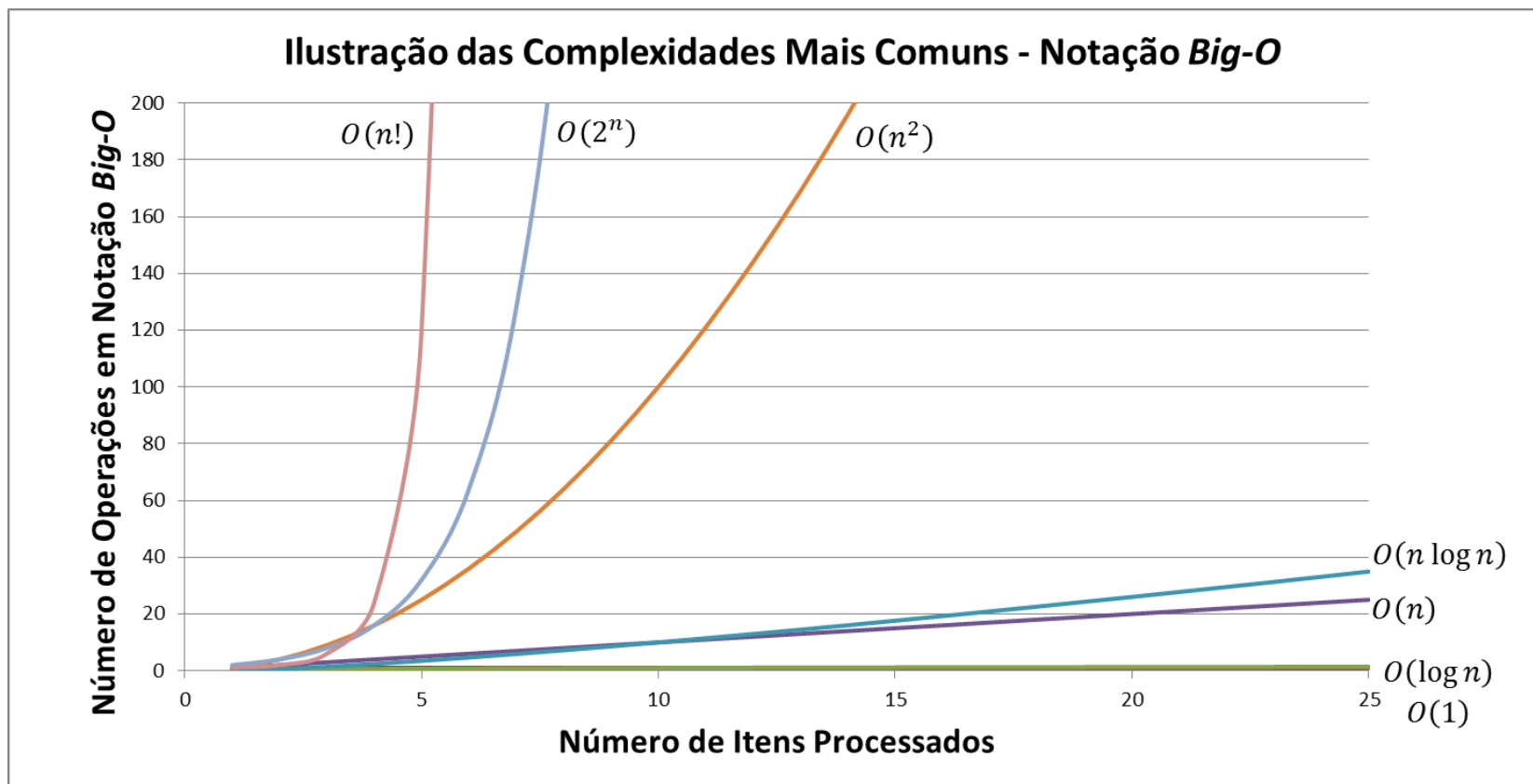
2. Crescimento de Funções

- Tipos comuns de funções:

- Constante: $O(1)$
- Logarítmica: $O(\log n)$
- Linear: $O(n)$
- Linearítmica: $O(n \log n)$
- Quadrática: $O(n^2)$
- Exponencial: $O(2^n)$
- Fatorial: $O(n!)$



2. Crescimento de Funções



- Vendo o gráfico qual tipo de algoritmo você escolheria para processar entradas muito grandes (por exemplo: com milhões de elementos)?

2. Crescimento de Funções

- Tempo de computação para $n = 1000$

running times	<i>time</i> ₁ 1000 steps/sec	<i>time</i> ₂ 2000 steps/sec	<i>time</i> ₃ 4000 steps/sec	<i>time</i> ₄ 8000 steps/sec
$\log_2 n$	0.010	0.005	0.003	0.001
n	1	0.5	0.25	0.125
$n \log_2 n$	10	5	2.5	1.25
$n^{1.5}$	32	16	8	4
n^2	1,000	500	250	125
n^3	1,000,000	500,000	250,000	125,000
1.1^n	10^{39}	10^{39}	10^{38}	10^{38}

Fonte: MANBER, U. *Introduction to Algorithms: A Creative Approach*. Boston: Addison Wesley, 1989.

Um algoritmo com tempo de execução exponencial irá gastar um tempo astronômico (bilhões e bilhões de anos) para resolver um problema com o tamanho da entrada $n = 1000$ (mesmo que a base esteja próxima de 1).



2. Crescimento de Funções

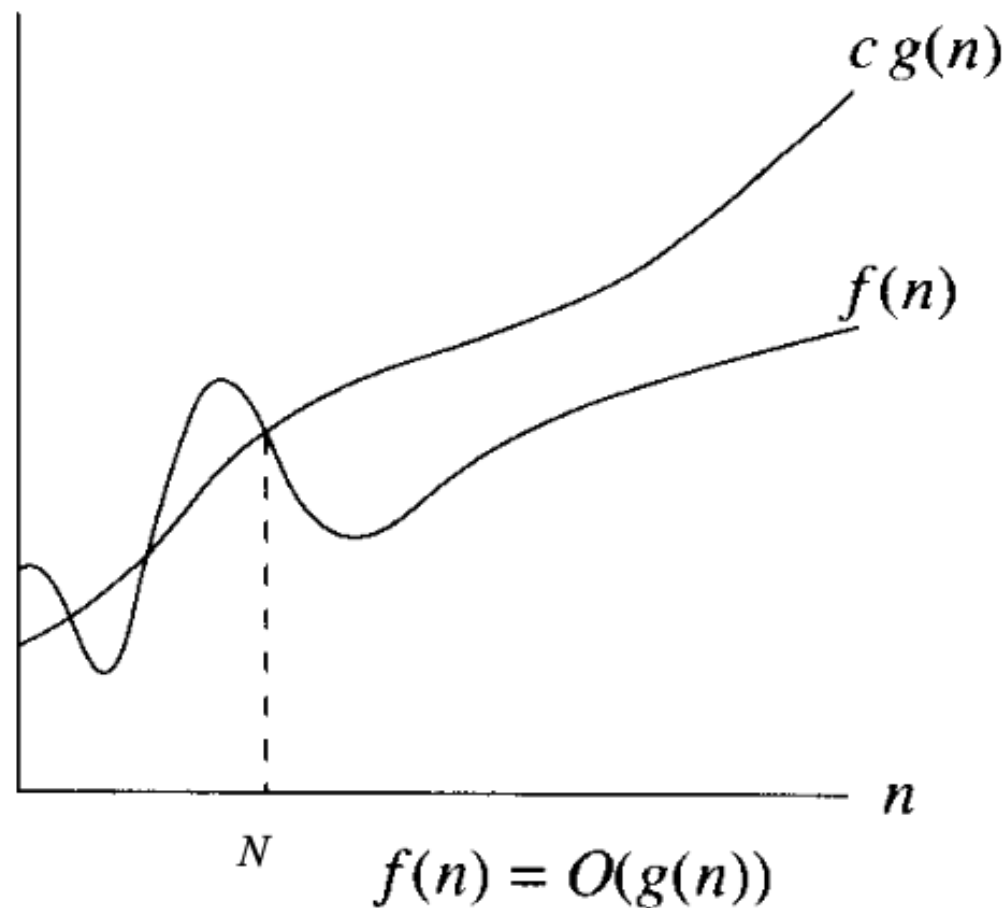
- Exemplos:
 - Acesso em lista: $O(1)$
 - Busca linear: $O(n)$
 - Busca binária: $O(\log n)$
 - Bubble Sort: $O(n^2)$

2. Notações assintóticas

- Notação O (Big- O) - Limite superior " $\leq$ "
 - Big- O representa o **limite superior** do tempo de execução de um algoritmo
 - Define o pior caso: crescimento **máximo** da função
 - Sejam f e g funções
 - **Definição:** $f(n)$ é $O(g(n))$ se e somente se existem constantes $c > 0$ e N tais que para todo $n \geq N$ nós temos $f(n) \leq c \cdot g(n)$.
 - **Observação:** na maioria das vezes escreveremos $f(n) = O(g(n))$ para significar que $f(n)$ é $O(g(n))$.

2. Notações assintóticas

- Notação O - Graficamente

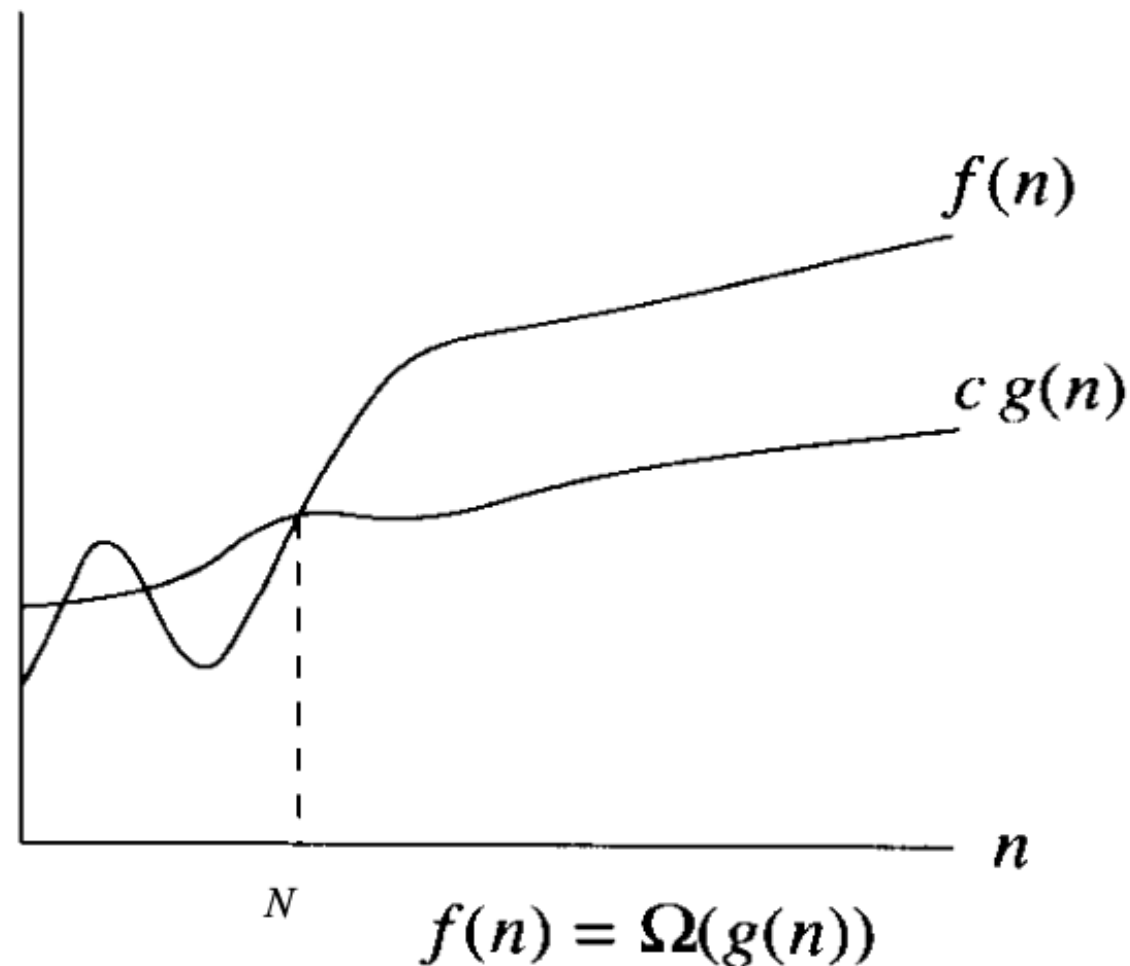


2. Notações assintóticas

- Notação Ω (Ômega) - Limite inferior “ $\geq$ ”
 - Representa o **tempo mínimo** de execução (melhor caso)
 - Define um **limite inferior**: não importa o quão otimizados estejam os dados
 - Sejam f e g funções.
 - **Definição:** $f(n)$ é $\Omega(g(n))$ se e somente se existem constantes $c > 0$ e N tais que, para todo $n \geq N$ nós temos $f(n) \geq c \cdot g(n)$.

2. Notações assintóticas

- Notação Ω - Graficamente



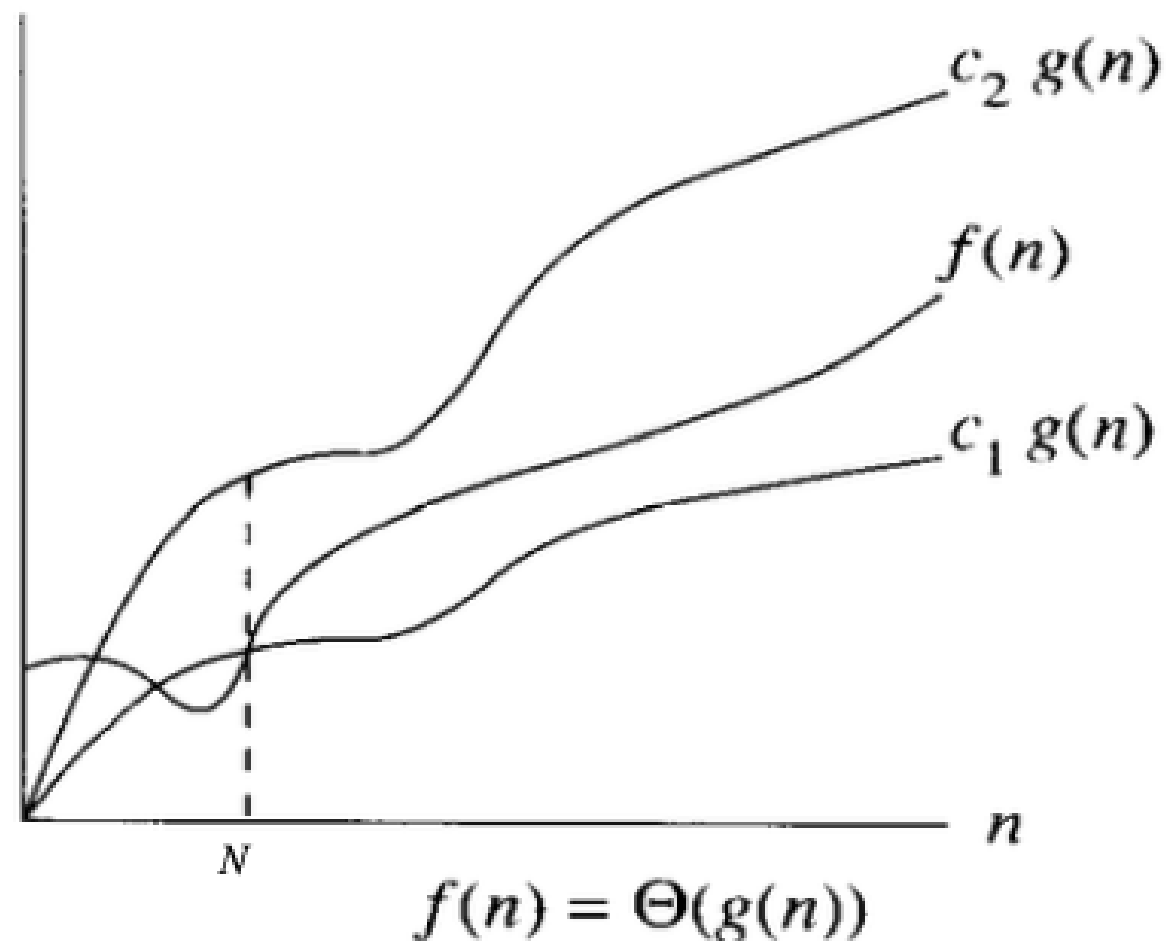


2. Notações assintóticas

- Notação Θ (Teta) - Crescimento exato “=”
 - Indica que o crescimento é limitado inferior e superiormente pela mesma função
 - Define o comportamento assintótico real
 - Sejam f e g funções.
 - **Definição:** $f(n)$ é $\Theta(g(n))$ se e somente se $f(n)$ é $O(g(n))$ e $f(n)$ é $\Omega(g(n))$.

2. Notações assintóticas

- Notação Θ - Graficamente





Observações sobre o material eletrônico

- O material ficará disponível na pasta compartilhada que é acessada sob convite
- O material foi elaborado a partir de diversas fontes (livros, internet, colegas, alunos etc.)
- Alguns trechos podem ter sido inteiramente transcritos a partir dessas fontes
- Outros trechos são de autoria própria
- Esta observação deve estar presente em qualquer utilização do material fora do ambiente de aulas do IFSP - Catanduva